

Name & Surname: Byron Broadway

Student Number: 05HA2312474

Name of your Support Centre: BELLVILLE



Code:

```
// -----  
// Automated Licence Compliance Management System (ALCMS)  
// -----
```

```
// 1. DATA INTEGRATION LAYER
```

```
// Simulated databases
```

```
const sabcLicenceDB = [  
  { id: "LIC001", idNumber: "8501015800083", name: "John Stemmet", paid: true, expiry: "2026-12-31" },  
  { id: "LIC002", idNumber: "9003025800085", name: "Thandi Nkosi", paid: false, expiry: "2025-06-30" },  
  { id: "LIC003", idNumber: "7812125800087", name: "Miriam Dlamini", paid: true, expiry: "2026-08-15" },  
];  
  
const retailTVSalesDB = [  
  { idNumber: "8501015800083", name: "John Stemmet", tvPurchaseDate: "2024-03-15", retailer: "Game" },  
  { idNumber: "9003025800085", name: "Thandi Nkosi", tvPurchaseDate: "2023-11-20", retailer: "Makro" },  
  { idNumber: "7812125800087", name: "Miriam Dlamini", tvPurchaseDate: "2025-01-10", retailer: "HiFi Corp" },  
  { idNumber: "[CREDIT_DEBIT_CARD_NUMBER]", name: "Lerato Molefe", tvPurchaseDate: "2025-05-22", retailer: "Incredible Connection" },  
  { idNumber: "9505055800081", name: "Michael Tau", tvPurchaseDate: "2024-08-30", retailer: "Game" },  
  { idNumber: "7001015800082", name: "Dept of Education", tvPurchaseDate: "2024-02-14", retailer: "Makro" },  
];
```

```
const homeAffairsDB = [  
  { idNumber: "8501015800083", name: "John Stemmet", address: "12 Main Rd, Soweto" },  
  { idNumber: "9003025800085", name: "Thandi Nkosi", address: "45 Church St, Pretoria" },  
  { idNumber: "7812125800087", name: "Miriam Dlamini", address: "78 Long St, Cape Town" },  
  { idNumber: "[CREDIT_DEBIT_CARD_NUMBER]", name: "Lerato Molefe", address: "23  
Vilakazi St, Johannesburg" },  
  { idNumber: "9505055800081", name: "Michael Tau", address: "9 Nelson Mandela Dr, Durban"  
},  
  { idNumber: "7001015800082", name: "Dept of Education", address: "222 Struben St, Pretoria"  
},  
];
```

// 2. GAP IDENTIFICATION ENGINE

```
function identifyNonCompliantHouseholds() {  
  const nonCompliant = [];  
  
  retailTVSalesDB.forEach((buyer) => {  
    const licence = sabcLicenceDB.find((lic) => lic.idNumber === buyer.idNumber);  
  
    if (!licence) {  
      // TV owner with NO licence at all  
      nonCompliant.push({  
        idNumber: buyer.idNumber,  
        name: buyer.name,  
        status: "NO_LICENCE",  
        tvPurchaseDate: buyer.tvPurchaseDate,  
        retailer: buyer.retailer,  
      });  
    } else if (!licence.paid) {
```

```
// Has licence but hasn't paid
nonCompliant.push({
  idNumber: buyer.idNumber,
  name: buyer.name,
  status: "UNPAID",
  licenceId: licence.id,
  expiry: licence.expiry,
});
} else if (new Date(licence.expiry) < new Date()) {
  // Licence expired
  nonCompliant.push({
    idNumber: buyer.idNumber,
    name: buyer.name,
    status: "EXPIRED",
    licenceId: licence.id,
    expiry: licence.expiry,
  });
}
});

return nonCompliant;
}
```

// --- 3. AUTOMATED COMMUNICATION MODULE ---

```
function sendNotifications(nonCompliantList) {  
  const notifications = [];  
  
  nonCompliantList.forEach((person) => {  
    const contact = homeAffairsDB.find((h) => h.idNumber === person.idNumber);  
    let message = "";  
  
    switch (person.status) {  
      case "NO_LICENCE":  
        message = `Dear ${person.name}, our records indicate you purchased a TV on  
${person.tvPurchaseDate} from ${person.retailer} but have no valid SABC TV licence. Please  
register and pay at https://tvlicence.sabc.co.za within 30 days to avoid penalties.`;  
        break;  
      case "UNPAID":  
        message = `Dear ${person.name}, your TV licence (${person.licenceId}) remains unpaid.  
Please make payment at https://tvlicence.sabc.co.za to avoid legal action.`;  
        break;  
      case "EXPIRED":  
        message = `Dear ${person.name}, your TV licence (${person.licenceId}) expired on  
${person.expiry}. Please renew at https://tvlicence.sabc.co.za.`;  
        break;  
    }  
  
    notifications.push({  
      recipient: person.name,  
      address: contact ? contact.address : "Unknown",  
      channel: "SMS + Email",  
      message: message,  
      sentDate: new Date().toISOString().split("T")[0],  
    });  
  });  
}
```

```
});  
});  
  
return notifications;  
}  
  
// --- 4. SELF-SERVICE PAYMENT PORTAL (Simulation) ---  
function processPayment(idNumber, amount) {  
  const licence = sabcLicenceDB.find((lic) => lic.idNumber === idNumber);  
  
  if (licence) {  
    licence.paid = true;  
    licence.expiry = new Date(new Date().setFullYear(new Date().getFullYear() + 1))  
      .toISOString()  
      .split("T")[0];  
    return { success: true, message: `Payment of R${amount} received. Licence renewed until  
${licence.expiry}.` };  
  } else {  
    // Create new licence  
    const newLicence = {  
      id: `LIC${String(sabcLicenceDB.length + 1).padStart(3, "0")}`,  
      idNumber: idNumber,  
      name: homeAffairsDB.find((h) => h.idNumber === idNumber)?.name || "Unknown",  
      paid: true,  
      expiry: new Date(new Date().setFullYear(new Date().getFullYear() + 1))  
        .toISOString()  
        .split("T")[0],  
    };  
    sabcLicenceDB.push(newLicence);  
  }  
}
```

```
    return { success: true, message: `New licence ${newLicence.id} created. Valid until  
    ${newLicence.expiry}` };
  }
}
```

```
// --- 5. COMPLIANCE DASHBOARD ---
```

```
function generateComplianceDashboard() {
  const totalTVOwners = retailTVSalesDB.length;
  const compliantOwners = retailTVSalesDB.filter((buyer) => {
    const licence = sabcLicenceDB.find((lic) => lic.idNumber === buyer.idNumber);
    return licence && licence.paid && new Date(licence.expiry) >= new Date();
  }).length;

  const complianceRate = ((compliantOwners / totalTVOwners) * 100).toFixed(1);
  const nonCompliant = identifyNonCompliantHouseholds();
  const estimatedRevenueLoss = nonCompliant.length * [CREDIT_DEBIT_CARD_CVV]; //
  R[CREDIT_DEBIT_CARD_CVV] annual TV licence fee

  return {
    totalTVOwners,
    compliantOwners,
    nonCompliantOwners: nonCompliant.length,
    complianceRate: `${complianceRate}%`,
    estimatedAnnualRevenueLoss: `R${estimatedRevenueLoss.toLocaleString()}`,
    nonCompliantBreakdown: {
      noLicence: nonCompliant.filter((p) => p.status === "NO_LICENCE").length,
      unpaid: nonCompliant.filter((p) => p.status === "UNPAID").length,
      expired: nonCompliant.filter((p) => p.status === "EXPIRED").length,
    },
  };
}
```

}


```
// -----  
// RUNNING THE SYSTEM  
// -----
```

```
console.log("=== SABC ALCMS - System Execution ===  
");
```

```
// Step 1: Identify non-compliant households
```

```
console.log("--- STEP 1: Gap Identification ---");  
const nonCompliant = identifyNonCompliantHouseholds();  
console.log(`Found ${nonCompliant.length} non-compliant TV owner(s):`);  
nonCompliant.forEach((p) => console.log(` • ${p.name} - Status: ${p.status}`));
```

```
// Step 2: Send automated notifications
```

```
console.log("  
--- STEP 2: Automated Notifications ---");  
const notifications = sendNotifications(nonCompliant);  
notifications.forEach((n) => console.log(` → Sent to ${n.recipient}: ${n.message}  
`));
```

```
// Step 3: Simulate a payment
```

```
console.log("--- STEP 3: Payment Processing ---");  
const payment = processPayment("[CREDIT_DEBIT_CARD_NUMBER]",  
[CREDIT_DEBIT_CARD_CVV]);  
console.log(` ${payment.message}`);
```

```
// Step 4: Generate dashboard
```

```
console.log("  
--- STEP 4: Compliance Dashboard ---");
```

```
const dashboard = generateComplianceDashboard();  
  
console.log(` Total TV Owners:    ${dashboard.totalTVOwners}`);  
console.log(` Compliant:          ${dashboard.compliantOwners}`);  
console.log(` Non-Compliant:      ${dashboard.nonCompliantOwners}`);  
console.log(` Compliance Rate:    ${dashboard.complianceRate}`);  
console.log(` Est. Revenue Loss:    ${dashboard.estimatedAnnualRevenueLoss}`);  
console.log(` Breakdown:`);  
console.log(`   - No Licence:      ${dashboard.nonCompliantBreakdown.noLicence}`);  
console.log(`   - Unpaid:          ${dashboard.nonCompliantBreakdown.unpaid}`);  
console.log(`   - Expired:         ${dashboard.nonCompliantBreakdown.expired}`);
```